



kakumeiteki

a fully-local coding & machine assistant — your files never leave your Mac

v1.1

MIT, free

zero dependencies

works offline

Install guide — macOS

10–15 minutes, all free software

What is this?

kakumeiteki (kaku for short) is a small AI agent that runs entirely on your own machine. It can explain and edit code, find duplicate files, detect and safely clean junk, and rename things across a project — using a local AI model, so nothing you work on is ever uploaded anywhere.

Honesty first: the AI model it uses is small and runs on your laptop, so it is **slow but real** — a task takes seconds to a few minutes. Every capability claim below is measured, not marketing. The harness (guard rails, tools, safety net) makes it reliable; the model sets the ceiling.

What you need

- A Mac (Apple Silicon like M1/M2/M3, 8 GB RAM or more)
- About 15 minutes and ~5 GB of free disk (for the AI model)
- Recommended terminal: **iTerm2** (free) — the built-in Terminal.app works but shows duller colours

Install — step by step

1. Install Node.js (version 20 or newer)

```
Go to https://nodejs.org and download the LTS installer, or with Homebrew:  
brew install node
```

2. Install Ollama (runs the AI model locally)

```
Go to https://ollama.com/download and install the macOS app, then open it once.
```

3. Download the AI model

```
ollama pull qwen3.5:4b
```

~3.4 GB download. If your Mac has only 8 GB RAM and feels sluggish, the lighter **qwen3:1.7b** (1.4 GB) is a measured, honest fallback — see the model table.

4. Get kaku

```
git clone https://github.com/leonardtudor11/kakumeiteki.git  
cd kakumeiteki
```

```
npm install -g .
```

5. Check everything

```
$ kaku doctor
✓ Node 25.9.0 (need ≥ 20)
✓ Ollama reachable at http://127.0.0.1:11434
✓ model "qwen3.5:4b" is pulled

all good — run: kaku
```

6. Run it

```
cd ~/some-project-folder
kaku
```

What you'll see

On start: the samurai splash, then a welcome card, then a bordered input box with a status bar underneath (folder · model · a live memory-of-conversation gauge · mode):

```
> type what you want done...

~/some-project · qwen3.5:4b 8.2k ctx · ctx 14% · 侍 build
```

Things to try first:

- *"explain what this project does"*
- *"find duplicate files in this folder"*
- *"scan for junk files that are safe to delete"* — it lists them and asks before touching anything

Every answer that changed files ends with a line kaku computes itself — the AI cannot fake it:

```
— verified 1/1 · node --test → exit 0 · changed: slugify.js
```

Is it safe? (the honest safety card)

- **It is locked to the folder you start it in.** It physically cannot read or write outside that folder. Widening the area requires you to type an explicit flag AND confirm.

- **It asks before every file change** (default mode) — you see a coloured before/after diff and answer y/N.
- **Everything is undoable.** Edits and deletions are backed up first; `kaku undo` restores them byte-for-byte. Deletions go to an undo store, never straight `rm`.
- **Dangerous commands are blocked** — `sudo`, pipe-to-shell and 30+ other classes are refused in every mode.
- **Your secrets stay secret** — `.env` files, keys and credentials are refused and redacted automatically.
- **Everything is logged** — an append-only audit file records what changed and whether you approved it.
- **No network** except talking to the model on your own machine. Nothing is uploaded, ever.

Which model? (measured, not vibes)

Scores are from the project's built-in test suite: 13 real tasks × 2 runs, on an M1 with 8 GB RAM.

Model	Download	Score	Speed	Verdict
qwen3.5:4b (default)	3.4 GB	24/26	~2 min/task	The brain. Best at everything except very large files.
qwen3:1.7b	1.4 GB	18/26	~1– 2 min/task	Surprisingly strong — the machine-assistant tasks (duplicates, junk, safe delete, rename) all pass because kaku's tools do the heavy lifting. Weaker at writing new code.
qwen2.5-coder:3b	1.9 GB	14/26	~30 s/task	Fastest, least reliable. Fine for quick edits and cleanup tasks.

Both smaller models score every machine-assistant task; where they fail is writing new code from a spec and multi-step reasoning. That is the honest ceiling of small local models — kaku will tell you when a result is UNVERIFIED rather than pretend.

```
# switch model for a session:
kaku --model qwen3:1.7b
```

Troubleshooting

Symptom	Fix
"Ollama not reachable"	Open the Ollama app once (menu-bar icon should appear), then re-run <code>kaku doctor</code> .
"model ... is not pulled"	Run the <code>ollama pull</code> command from step 3.
First answer takes ~1 min	Normal — the model loads into memory on first use, later turns are faster.
Whole Mac feels slow while it thinks	Normal on 8 GB — the model uses most of the RAM while answering. Try <code>qwen3:1.7b</code> .
Banner looks wrong / weird colours	Use iTerm2, or start with <code>KAKU_PLAIN=1 kaku</code> for plain mode.

Uninstall (clean, complete)

```
npm uninstall -g kakumeiteki
ollama rm qwen3.5:4b          # removes the model
rm -rf ~/.kakumeiteki        # removes sessions, undo backups, audit logs
```

Windows UNTESTED

kaku is built on portable Node.js and should run, but **nobody has tested it on Windows yet** — you would be the first. If you try: use **Windows Terminal** or the VS Code terminal (NOT the old cmd.exe), install Node 20+ and Ollama for Windows, then the same steps in PowerShell. Please report anything broken.

kakumeiteki v1.1 · MIT license · github.com/leonardtudor11/kakumeiteki · every capability claim in this guide is measured by the repo's eval suite ([eval/scorecard.md](#))